从热度到驻留。

Hash Join 内存调度的协作优化

数据集如何逼近真实问题

可见访问如何转化为保留价值

R85_fast_prior8 如何作出每一次选择

观测 → 估计 → 估值 → 执行

Vanius0525 × cyanneko

基于代码与归档实验的技术叙事

正文 15 节 · 含公式、流程与结果

01 / 阅读起点

把有限内存留给真正有价值的数据

我们优化的不是 Hash Join 的连接逻辑，而是它在内存不足时的整分区驻留与搬运决策。

多个算子共享同一块内存。BUILD 持续增加分区体积；PROBE 请求到来时，目标若在磁盘，必须完整读回。每一次选择受害者，都在“现在写出去多少”与“以后还要搬回来多少”之间权衡。

R85
本报告主讲版本 · fast_prior8

4.087790
当前完整主测最高综合分

12 + 72
动态与静态两组 · 各占50%

截至这份快照，**R85_fast_prior8** 在同学 R85 归档的八方法同场比较中综合分最高。它把算子活跃度与算子内分区偏好分开估计：前者快速跟随热点迁移，后者用更长历史稳定判断。

本报告回答什么

数据为何这样设计？哪些优化改变了决策？当前算法怎样把有限观测变成驱逐集合？哪些负结果改变了我们的判断？

如何理解“当前最优”

这里指固定主测上的最高实测均分。R85 比 R81 高 0.003361 分，静态组略退步；仓库正式默认入口仍为 R75。

阅读路线

问题与观测 → 数据与演进 → 公式与执行 → 结果与认识

本轮只整理代码与已有实验，不新增训练或性能测试。数据快照为 dyna1.0 与 final2.1；final3.0 单独作为敏感性观察。

02 / 问题与目标

优化对象是成本，不只是 Miss

一次小分区 Miss 和一次大分区 Miss，可能相差几个数量级的搬运费用。

真实计分成本

$$C = \frac{B_{read}}{R} + \frac{B_{write}}{W} + \lambda \cdot N_{miss} + D$$

R、W：读写带宽（字节/μs）；B：实际搬运字节；λ：输入中的真实Miss罚值；D：回调决策时间。

降低 Miss 有价值，但要同时看读写字节。我们可以让多个便宜的冷请求 Miss，却避免一个昂贵分区反复被逐出；反过来，盲目保护大分区也可能挤走大量更常用的小分区。

三个独立的量	它决定什么	策略应该怎样使用
分区大小 s	容量占用、读写成本	进入搬运成本和装填约束
PROBE 概率 p	未来驻留收益	从合法观测估计，不由大小推断
BUILD 增长	后续内存压力	影响保留机会，不等同于未来热度

调度器可以改变什么

合法动作为整分区 `SPILL` 与 `LOAD`。调度器不能拆热点分区、换哈希函数、压缩数据、重排请求或把查询改成批处理。冷请求也不能被拒绝准入。

线上估值代理

$$\tilde{\lambda} = 100\ \mu s$$

真实λ不在SystemState中；100只用于算法内部估值，评分始终使用各输入自己的真实λ。

最关键的转向
从“哪块数据看起来最冷”，转向“释放足够空间时，牺牲哪个集合的总成本最低”。

03 / 观测边界

没有被看见，不等于没有被访问

普通 Hit 常常不暴露身份；Miss 一定暴露目标。直接累计可见访问，会把调度策略造成的缺页当成真实热度。

信息来源	可获得的信息	在当前算法中的用途
累计 Hit / Miss	总请求进度，无完整Hit身份	构造PROBE逻辑时钟 q
periodic 与 Miss 回调的 next	下一条请求的算子与分区	去重采样，更新概率估计
当前 Miss 与驱逐后重访	哪个盘上分区被再次需要	更新重访风险与历史回退
分区大小、阶段、位置	当前可合法操作的状态	成本计算、容量与阶段约束

时钟与去重

$$q = Hit + Miss，\quad next\ 的序号 = q + 1$$

同一个next可能在多个回调中出现；只接收序号大于已采样序号的请求。连续两次同key但序号不同，仍是两个样本。

为什么不只统计 Miss

驻留成功的热点少产生Miss，反而不再可见。Miss更适合描述“驱逐后损失”，不能直接替代无偏访问频率。

为什么保留 next 样本

它在部分Hit之前就暴露请求，能补足只看Miss的偏差。固定独立请求下更易解释；动态相关流仍需衰减与稳健估计。

实现与统计解释分开

R85 将 periodic 和 Miss 后暴露的 `next` 合并去重后采样，没有为两类样本设置独立权重。当前 Miss 不直接再加入这条概率采样流，但会更新重访与历史状态。这个区分避免了简单重复计数，也保留了周转反馈。

256 / 2048按PROBE序号计时。概率状态对齐到t=max(q, last_next_ordinal)，已暴露next时可为q+1；重访风险仍按q。二者都不是墙钟时间。

04 / 数据设计 · 静态

让测试逐渐对准真正的问题

数据集不只是更多案例。每次迭代都在修正一种错误的选型信号。

起点 / 综合集 2.0

从少数成员数据，走向统一评测

500 train、50 eval。统一成本与评分口径，减少在各自擅长的数据上分别报优的偏差；其中仍混有均匀独立请求。

聚焦 / 综合集 2.1

把规模、容量压力与倾斜放到主线

推荐540 train、72 eval；以hotspot、Zipf和lognormal为主。均匀对照及特殊边界移出主测；我方从推荐训练集固定train432/dev108，并选择60例训练面板。

追问 / 综合集 3.0

全局倾斜，是否真的意味着分区热度可学习

保持父规格和BUILD种子，重建全部PROBE。显式保证每个算子内部也有概率差异，防止“只有算子冷热不同，算子内仍接近均匀”。540 train、72 eval。

三种随机偏斜，覆盖不同形状

Hotspot

少量热门分区，配合冷长尾。检验热点保留与冷请求周转。

Zipf

访问概率按秩衰减。检验多层次热度与容量装填的相互作用。

Lognormal

连续、非规则的概率差异。减少策略只适配简单头部结构的机会。

3.0 的偏斜度验收

$$\mathrm{TV}(p, \mathrm{uniform}) = \tfrac{1}{2} \sum_j \left| p_j - \frac{1}{P} \right|$$

TV为总变差距离，越大越偏离均匀。目标分布要求≥0.20，实际采样频率要求≥0.10；这是数据设计阈值。

均匀请求没有可学习的分区概率差异，但大小和搬运成本仍可不同。移出主测是任务聚焦，并不意味着这类调度问题必然简单。

05 / 数据设计 · 动态

稳定倾斜之外，还要追踪热点迁移

静态数据回答“能不能识别热点”；动态数据追问“热点变了，多久才能跟上”。

12
dyna1.0 完整案例

500万
每例操作数

3 × 2
迁移速度与阶段布局组合

设计维度	变化方式	检验的能力
Slow / Fast / Burst	缓慢变化、快速切换、突发热点	估计窗口的响应与稳定性
Shared / Staggered	阶段同步或错开	跨算子竞争；错开时可出现BUILD与PROBE共存
长操作流	每例500万操作	累计周转损失与决策开销

主测怎样汇总

当前主榜分别计算 dyna12 与 final2.1/72 的逐例均分，再各赋50%权重。这样不会仅因静态案例更多，就把动态适应能力稀释掉。

补充面板的角色

balanced24把static12与dyna12配成等权回归；inde提供独立补充族与更长请求。它们帮助定位问题，不自动改写主榜口径。

仍值得补齐的设计

正交改变分区大小与访问概率的正相关、独立、负相关；加入超大冷请求和真实阶段交叠，进一步检验整集合装填。

固定身份，比记住文件名更可靠

新版会重编号，旧案例要按父谱系映射，而非按文件名猜测。我们固定输入指纹、生成种子、数据版本与结果所属批次。3.0已进入R86补充实验；它的成绩单独展示，不与2.1混成一条进步曲线。

06 / 优化路线

从独立尝试，到机制层面的融合

有效的进步，逐渐从“加入更多规则”转向“更好地估计，同一套合法动作的价值”。

01 / 我方 paired100

固定动作规划，比较两个方案

把next样本按序号去重，用100μs代理与短窗口未来代价比较驱逐集合。小树、候选池扩展及校准没有形成稳定收益，AI线随后暂停。

02 / 双方风险探索

把分区自身与算子共性联系起来

同学R74/R75发展驱逐后的重访风险和算子共享；我方joint另用盘上暴露与意外Miss校准方案比较的未来均值。动态有收益，但没有稳定通过原训练面板。

03 / R79 → R80

将next概率估值接入原调度框架

保留同学的合法规划器、重访与扫描分支；吸收我方next去重、64步价值估计、100μs代理及动作差额比较思想，形成hierarchy与paired两个变体。

04 / R81

将算子热度与分区偏好分开计时

算子活跃度用256半衰期快速跟踪；算子内条件分布用2048半衰期积累稳定证据。普通随机路径使用两者的乘积。

05 / R85_fast_prior8

校准快窗口中的先验强度

缩短窗口也减少了累计样本质量。将快算子先验总量从64降为约8.009，避免快窗口被同样大的均匀先验过度拉平。

融合的准确含义

R85继承的是双方若干有效机制。它没有完整合并我方joint校准，也没有实现显式热点核心。其当前配置只生成一个hierarchy方案，保留的双方案比较代码不在这条主路径执行。

07 / 算法总览

一条完整的决策链

先确认状态，再估计请求价值，最后在有限候选内解决容量约束。

1 **接收回调并更新状态**

Overflow / Miss / Periodic；确认上次动作是否生效，维护PROBE序号与去重next。

↓

2 **识别可用证据与请求形态**

维护快算子、慢分区样本；检测已被连续观测支持的扫描或等大小循环。

↓

3 **普通PROBE：两层热度合成**

快速算子边际 × 自适应收缩的慢分区条件概率，得到每个分区的p̂。

↓

4 **计算单位空间保留价值**

64p̂ ×（读单位成本 + 写单位成本 + 100/大小）；对明确next施加强保护。

↓

5 **选择能覆盖缺口的驱逐集合**

最低优先级top4枚举15个非空子集；不够时走线性单受害者或堆回退。

↓

6 **剪掉冗余SPILL，完成必要LOAD**

Miss目标必须加载；周期预取受空闲容量/已确认循环条件限制；记录动作意图。

特殊分支：确认的等大小全局循环可直接选择近期对象；全局scan信号会停用hier估值，匹配目标算子才用扫描距离，其余走历史/风险。BUILD也走历史回退。

> **职责分离**
> 估计器回答“未来可能用到谁”；规划器回答“要释放这些字节，牺牲哪组对象更合适”。概率再准确，也不能跳过整分区的组合约束。

08 / 核心公式 · 两层估计

算子跟得快，分区看得稳

记 K 为算子数，P 为每算子的分区数。对每个去重next样本，分别累计快算子质量 F_i 与慢分区质量 m_{ij}；$M_i=\Sigma_j m_{ij}$。

① 时间衰减

$$F_i \leftarrow F_i \cdot 2^{-\Delta q/256}, \quad m_{ij} \leftarrow m_{ij} \cdot 2^{-\Delta q/2048}$$

每个去重next样本对应的算子与分区分别加1。其他状态延迟衰减，访问时再对齐到当前逻辑时钟。

② 快算子边际

$$\hat{\pi}_i = \frac{F_i + \beta/K}{\sum_k F_k + \beta}$$

先回答：下一次请求来自哪个算子？

③ 慢分区条件概率

$$\hat{\theta}_{ij} = \frac{(1-\alpha_i)m_{ij} + \alpha_i M_i/P + 64/(KP)}{M_i + 64/K}$$

再回答：在这个算子内部，哪个分区值得保留？α决定向算子内均匀分布收缩多少。

$$\hat{p}_{ij} = \hat{\pi}_i \cdot \hat{\theta}_{ij}$$

全局访问概率 = 算子活跃度 × 算子内分区偏好

④ R85 的先验校准

$$\beta = 64 \cdot \frac{1 - 2^{-1/2048}}{1 - 2^{-1/256}} \approx 8.009479828$$

指数累计的稳态质量与1/(1−衰减系数)成正比；快窗口质量更小，因此同步缩小快先验。慢条件先验总量仍为64。

这避免了一个隐藏混杂：如果只缩短窗口却保留64的先验，变化的不仅是响应速度，还有先验相对于数据的强度。校准修正这一比例，并不增加新的观测。

09 / 核心公式 · 自适应共享

证据不足时共享，差异清晰时分开

同一个算子的分区有共性，也可能极不相同。共享强度应该随证据变化。

维护算子慢样本总质量 M_i、分区质量平方和 $S_i=\Sigma_j m_{ij}^2$，以及样本权重平方和 $Q_i=\Sigma_e w_e^2$。

① 有效样本量

$$n_{eff,i} = \frac{M_i^2}{Q_i}$$

用于修正加权采样噪声；它不是回调次数，也不是全部真实访问数。

② 扣除有限样本噪声后的差异

$$\nu_i = \max\left(0, \frac{P\,S_i/M_i^2 - 1 - (P-1)/n_{eff,i}}{1 - 1/n_{eff,i}}\right)$$

③ 证据强度与共享比例

$$\kappa_i = \nu_i \cdot \frac{M_i - 8}{M_i + P}, \quad \alpha_i = \frac{0.75}{1+\kappa_i}$$

当 $M_i \le 8$、有效样本量≤8，或数值条件不足时，源码使用默认 $\alpha=0.75$。证据越多且分区差异越大，α 越小，越相信分区自己的质量；近均匀时保留较强共享。

少量观测

一个分区暂时没出现，很可能只是没抽到。收缩可以避免过早把它判为冷。

明确倾斜

如果少数分区持续占据大多数样本，过强共享会抹平热点。自适应α让差异逐渐显现。

共享发生在两个层面

π跟踪算子之间的资源需求；α控制算子内部的分区平滑。前者解决“哪台算子更忙”，后者解决“忙碌是否集中在少数分区”。

10 / 核心公式 · 成本与集合

概率只是起点，还要算每个字节的价值

对大小为 s_{ij} 的普通PROBE分区，R85把概率转为未来64次访问的复用质量。它可以超过1，不是单次复用概率。

① 单位空间保留优先级

$$\rho_{ij} = 64\hat{p}_{ij} \cdot \left(\frac{1}{R} + \frac{1}{W} + \frac{100}{s_{ij}} \right)$$

先按ρ从小到大找可牺牲对象；同值时优先更大对象。next目标额外加10^{12}作为强保护。

② 前四个低优先级对象内枚举集合

$$\mathcal{L}(S) = \sum_{(i,j)\in S} \left[\rho_{ij} s_{ij} + 0.25 \cdot \frac{s_{ij}}{W} \right]$$

约束Σs ≥ 当前缺口。q>0或R≥W时进入top4，枚举最多15个非空子集。

乘回大小后，第一项成为预计读入、再次写出与Miss代理的总损失；第二项加入当前写出的权重。**0.25是当前启发式系数**，不是物理写费只收25%。改为1的既有尝试没有稳定获益。

为什么不只保留大对象

大对象一次搬运贵，也占更多空间。单位空间收益与整个集合的装填必须同时考虑。

top4放不下怎么办

已有PROBE时先检查最低优先级单个对象，再用堆取出更多对象；尚无PROBE时全排序贪心。最终剪除冗余SPILL。

一个数值例子

π=0.4、θ=0.15，则p̂=0.06。若分区4 MiB、R=1024 B/μs、W=512 B/μs，64步搬运与Miss代理约47.57 ms；单位空间约11.89 ms/MiB，另计0.25倍当前写入项。

候选数n≤KP：常见路径约O(n)，多受害者回退O(n+v log n)，最坏O(n log n)；辅助状态O(KP)。

11 / 执行细节

BUILD、Miss 与预取，各有自己的职责

回调	当前行为
onMemoryOverflow	更新证据，排除当前INSERT目标，按缺口选择SPILL并剪除冗余动作。
onPartitionMiss	结束目标的盘上重访观察；释放足够空间后，必须LOAD目标整分区。
onPeriodicAdjust	首个Overflow/Miss启用调度器后才更新next并考虑预取；此前直接返回。

预取：明确会用，还要考虑是否额外驱逐

若目标已在内存、不是PROBE阶段或大于总内存，就不预取。空闲空间足够时可LOAD；若需要额外驱逐，只有已确认的等大小循环分支会继续。这样保留普通Miss回调带来的后续信息。

重访风险：为历史与特殊分支保留后备信息

盘上暴露与事件衰减

$$E \leftarrow E\,e^{-\Delta q/\tau} + a\tau(1-e^{-\Delta q/\tau}),\quad N \leftarrow N\,e^{-\Delta q/\tau}$$

$$\mu = (E+64)/(N+1),\quad r = 64/(64+\mu)$$

a为正在盘上观察的分区数；确认驱逐后开始观察，Miss计事件，预取读回只结束暴露。分区τ=512、算子τ=128，是时间常数而非半衰期。

在历史回退分支，周期覆盖率不足0.75时，可见历史权重为0.25，其余结合分区/算子重访风险；覆盖率≥0.75时权重切为1，完全采用可见历史。普通随机PROBE主路径直接使用层次概率，**并不是把风险分数再叠加一次**。

BUILD：先后顺序改变机会成本

已确定会在BUILD内落盘的分区

首次写出 a + 后续追加 b = 最终分区大小

提前落盘不额外增加这部分累计写字节；改变的是占用内存的时间、挤走谁，以及谁能留到PROBE。

当前R85没有新的BUILD增长预测器；BUILD候选沿用历史/风险分支。降低BUILD当前写量也可能增加之后的PROBE周转，因此没有另加“BUILD不罚Miss就优先牺牲”的规则。

12 / 实验结果

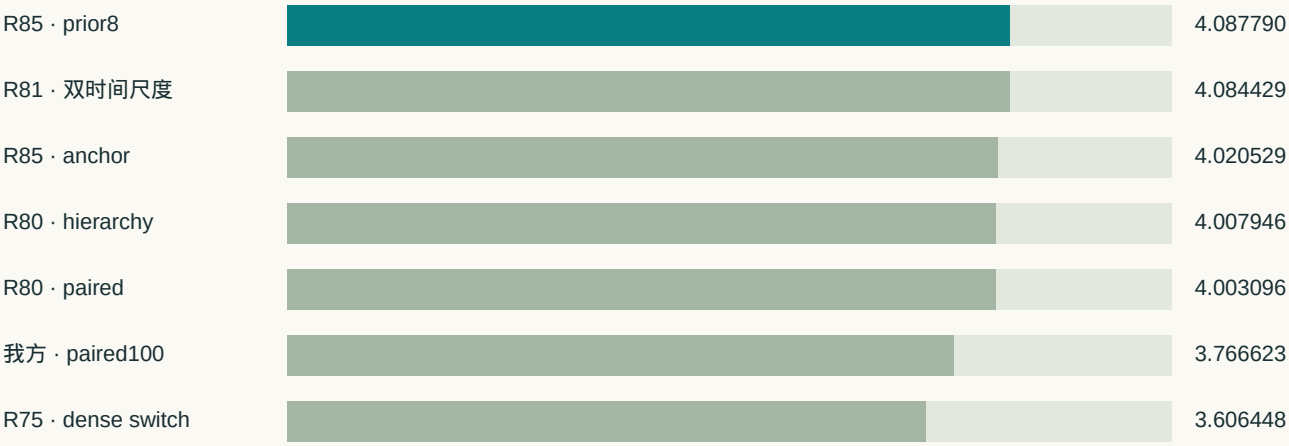
进步主要来自更好的概率估值

下图来自**同一R85归档批次**：八方法含Naive、单次CPU评测；dyna1.0/12与final2.1/72分别均分，再各占50%。分数越高，表示相对同批Naive节省的成本越多。

评分与汇总

$$score_x = clamp(10 \cdot (1 - C_x/C_{Naive,x}), 0, 10)$$

$$综合分 = ½ \cdot mean(score_{dyna}) + ½ \cdot mean(score_{final})$$



主线方法	dyna12	final2.1 / 72	综合
我方 paired100	4.049627	3.483619	3.766623
R80 hierarchy	4.306697	3.709196	4.007946
R81 双时间尺度	4.455848	3.713010	4.084429
R85 fast_prior8	**4.482767**	**3.692813**	**4.087790**

读懂增益来源：R85相对R81，动态组+0.026919，静态组−0.020197，综合仅+0.003361。最终静态组有6例成本高于Naive；平均收益没有消除长尾问题。

final3.0的R86补充中，R85A静态分4.087043，R85 anchor为4.168410，排序发生变化。它们是不同请求版本，不能把新旧分差当作算法提升。

13 / 关键认识

五个真正改变方向的教训

01 候选更丰富，不保证选得更好

covering-singleton和小树扩展没有稳定晋升。好动作进候选池之后，仍可能被有偏估值排错；因此先改善观测与估值，再扩搜索。

02 减少 Miss，可能付出更大的读写代价

R12的train519比R80少833次Miss，按该例真实罚值只省0.099127秒，却多7.346876秒模拟IO费用，净物理成本增加约12.63%。软核心的直觉需要周转字节验证。

03 改成总价值排序，会改变整个可行集合

R10 value_rank在固定训练面板最坏单例物理成本约为R80的23.565倍。成本×概率有用，但移除单位空间归一化，会改变候选入口和整分区装填。

04 一个数据域的优势，不等于普遍优势

我方joint在动态12例物理成本全部改善；原72例均分仅比paired高约0.000085，最坏物理成本增加约5.48%。需要同时看动态适应与静态长尾。

05 更低的当前写量，不一定是更好的BUILD策略

既有cold-BUILD尝试累计减少约0.409秒BUILD写出，却增加约12.393秒PROBE读写。阶段之间的机会成本，比眼前最小写量更值得关注。

> **当前最有用的判断标准**
> 总成本是否下降？下降来自读、写、Miss还是决策时间？最坏案例发生了什么？这些问题比“机制听起来是否合理”更能推动优化。

本页数字来自各自固定训练/评测报告，用于解释机制；不与上一页的同场主榜合并计分。

14 / 补充公式 · 回退分支

当请求结构比频率更有用

以下是R85保留的历史、风险与扫描分支。普通随机PROBE使用前述层次概率；当分支条件变化时，仍用同一个成本与容量规划器执行。

① 可见历史：次数与最近性

$$h_j \leftarrow h_j \cdot \frac{256}{256+\Delta q} + 1$$

$$u_j = \frac{\min(1,\ h_j/16)}{1 + (\max(0,\ q+1-last_j)+1)/64}$$

历史只从可见当前Miss或next更新；此处是有理式衰减，不是概率层的指数半衰期。

② 周期观测覆盖率

$$G \leftarrow G\, e^{-(t-t_G)/256} + (1-e^{-1/256})$$

$$c = \min\left(1,\ \frac{G\, e^{-\max(0,q-t_G)/256}}{1-e^{-\max(1,q)/256}}\right)$$

只有periodic暴露新的PROBE next时，才增加覆盖质量G。t为新next序号；随后令t_G=t。

③ 回退估值的组合

$w = 1$（$c \geq 0.75$），否则 $w=0.25$

$$reuse = w \cdot u + (1-w)[(1-\alpha)r_{partition} + \alpha r_{operator}]$$

仅在有相应重访记录时使用风险项；没有分区记录则回退到u，没有算子记录则只用分区值。

结构分支	触发与行为
扫描	连续同算子、相同非零步长至少6次，且可见观察密度超过一半；匹配目标算子时用1−步距/P替换u，再进入风险/覆盖混合。
等大小全局循环	所有算子PROBE、所有分区等大且非空；每条后继边置信计数≥3并构成覆盖全分区的单环。扫描近期对象，排除当前目标和next，寻找单个可覆盖缺口的对象。

这些是保留的结构特化，不是本轮主要增益来源。模式证据不足时继续走通用流程，不需要假设请求一定循环或扫描。

15 / 版本与复现

让这份叙事有明确的落点

主讲版本固定，数据口径固定。后续同学更新算法时，可以从这个快照继续讲清变化。

R85_fast_prior8 · 纯C++14调度器

源码提交：4a259b7b1b4232f521decdb22b826e41d16baf48

路径：src/experiments/r85_math_ablations_20261030/R85_fast_prior8.cpp

SHA256：a0ebc08399a8cd8c039dae59a779231df0c35306af3ea3269df3067c321779b5

查看固定版本源码 ↗

在第三方环境重测

将仅含 `MySpillScheduler` 的源码接入同一官方框架，使用C++14与O2编译；保持输入指纹一致，每方法每题一次，Naive同批重测。保存IO、Miss、决策时间与失败，按两套数据各50%汇总。100μs只属于算法估值，不改测试输入的罚值。

主要阅读依据

[1] R85 完整结果与数学消融：当前主讲版本、完整两组成绩、先验校准与anchor比较。

[2] R85_fast_prior8.cpp：公式与实际控制流的直接依据，尤其MODE=2、top4规划、预取与回退。

[3] R80 机制迁移报告：我方paired思想与同学原框架的连接。

[4] final2.1、final3.0与dyna1.0：数据设计、版本与生成配方。

[5] 我方Round10、Round12、完整72例：集合排序、软核心与联合校准的正负结果。

[6] R86 补充敏感性实验：将新final3.0与旧主测分开理解。

本报告对源码分支与公式作静态核对，成绩引用固定归档；不是一次新增复测。源码链接是否可打开取决于原仓库访问权限，网站正文和PDF可独立阅读。制作：Vanius0525 × cyanneko 项目资料整理。